

Подпрограммы на NASM

Что это

В ассемблере NASM процедуры объявляются как обычные метки в коде, а управление ими осуществляется командами `call` и `ret`. В отличие от MASM/TASM, в NASM нет специальных директив вроде `PROC` и `ENDP`

Синтаксис

```
my_procedure:
```

```
    ; Код процедуры
```

```
ret ; Возврат в место вызова
```

Вызов процедуры

```
call my_procedure
```

Передача параметров и соглашения

Способ передачи параметров зависит от разрядности системы (x86 или x64).

Режим x86

Параметры передаются **через стек** в обратном порядке (справа налево). Очищает стек вызывающий код (cdecl)

Режим x86

```
section .text
global _start
_start: push 5 ; Второй аргумент
        push 10 ; Первый аргумент
        call sum; Вызов функции
        add esp, 8 ; Очистка стека (2 аргумента по 4 байта)
        ; Выход из программы...

sum:
        push ebp ; Сохраняем старый указатель кадра
        mov ebp, esp ; Создаем новый кадр стека
        mov eax, [ebp + 8] ; Первый аргумент (10)
        add eax, [ebp + 12] ; Второй аргумент (5)
        mov esp, ebp ; Восстанавливаем стек
        pop ebp ; Восстанавливаем указатель кадра
        ret ; Результат возвращается в EAX
```

Режим x86

Адрес памяти	Что лежит внутри стека	Кто это туда положил
«ESP + 8»	Число 5 (Второй аргумент)	Вызывающий код (push 5)
«ESP + 4»	Число 10 (Первый аргумент)	Вызывающий код (push 10)
ESP	Адрес возврата (Куда вернуться после ret)	Команда call автоматически

Кадр стека

— это блок данных в оперативной памяти, который автоматически создаётся при вызове функции. Он хранит информацию, необходимую для её работы, и существует ровно до тех пор, пока функция выполняется

Кадр стека

Каждый раз, когда программа вызывает новую функцию, в память добавляется новый кадр, содержащий:

- **Аргументы:** данные, переданные в функцию извне.
- **Адрес возврата:** точка, куда программа должна вернуться после завершения функции.
- **Локальные переменные:** временные данные, созданные внутри этой функции.

1. Создание кадра стека (Пролог функции)

```
push ebp          ; Сохраняем старый указатель кадра  
mov ebp, esp      ; Создаем новый кадр стека
```

`push ebp`: Регистр EBP указывает на базовый адрес кадра стека предыдущей функции. Мы сохраняем его в стек, чтобы не испортить. Стек сдвигается еще на 4 байта вниз.

`mov ebp, esp`: Мы копируем текущее значение указателя стека ESP в EBP. Теперь EBP жестко зафиксирован на этой позиции. Это наша «точка отсчета» внутри функции.

1. Создание кадра стека (Пролог функции)

- $[EBP]$ указывает на старое значение EBP.
- $[EBP + 4]$ содержит адрес возврата.
- $[EBP + 8]$ содержит первый аргумент (10).
- $[EBP + 12]$ содержит второй аргумент (5).

2. Чтение аргументов

```
mov eax, [ebp + 8]      ; Первый аргумент (10)
add eax, [ebp + 12]     ; Второй аргумент (5)
```

Поскольку EBP зафиксирован, мы всегда знаем точное смещение аргументов.

Мы берем число по адресу EBP + 8 (это 10) и кладем в регистр EAX.

Затем прибавляем к EAX число по адресу EBP + 12 (это 5).

Результат (15) остается в EAX. По соглашению, функции возвращают результат именно через регистр EAX.

3. Удаление кадра стека (Эпилог функции)

```
mov esp, ebp      ; Восстанавливаем стек  
pop ebp           ; Восстанавливаем указатель кадра
```

`mov esp, ebp`: Мы возвращаем указатель стека ESP на то место, где зафиксирован EBP. Очищая таким образом любые локальные переменные, если бы мы их создавали внутри функции.

`pop ebp`: Извлекаем из стека сохраненное ранее старое значение EBP обратно в регистр.

Сохранение регистров

Внутри процедуры хорошим тоном считается сохранять регистры, которые вы изменяете. Делается это с помощью `push` в начале и `pop` в обратном порядке в конце

```
my_func:
```

```
    push ebx                ; Сохраняем ebx
```

```
    push ecx                ; Сохраняем ecx
```

```
    ; ... полезный код, где используются ebx и ecx ...
```

```
    pop ecx                 ; Восстанавливаем ecx
```

```
    pop ebx                 ; Восстанавливаем ebx
```

```
    ret
```

Режим x64

регистры: RDI, RSI, RDX, RCX, R8, R9.

```
section .text
global _start

_start:
    mov rdi, 10          ; 1-й аргумент
    mov rsi, 5           ; 2-й аргумент
    call sum_x64

    ; Результат в RAX равен 15

sum_x64:
    mov rax, rdi          ; RAX = 10
    add rax, rsi          ; RAX = 10 + 5
    ret
```

Экспорт процедур

Если процедура должна быть доступна из других файлов (например, из кода на C/C++), её нужно объявить с директивой `global`:

```
global my_func
```

```
my_func:
```

```
    ; код процедуры
```

```
    ret
```

Задание

- Создать процедуру, возводящую в квадрат число введенное пользователем
- Разработать процедуру, вычисляющую значение по формуле: $b^2 - 4acs$